

TECSUP · Carrera de Big Data y Ciencia de Datos

FUNDAMENTOS DE GESTIÓN DE DATOS

UNIDAD 1

FUNDAMENTOS DEL DATO Y SQL

Material de estudio · Semanas 1 a 4
Del dato al modelo relacional

Semana	Tema
1	El Dato y la Gestión de Datos
2	SQLite Básico: Consultas y Filtros
3	SQL Avanzado: JOINS y Agrupaciones
4	Modelado Relacional

Docente: Pilar Rocío Sayán Mejía
Periodo académico: 2026-II

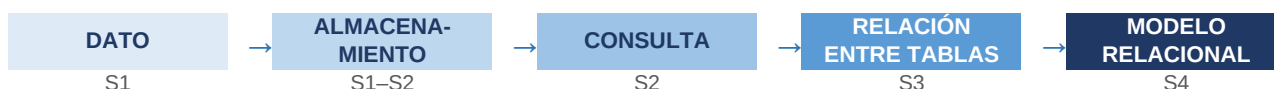
Contenido

Introducción a la Unidad 1.....	3
¿Qué aprenderemos?.....	3
Caso transversal: RutaMarket.....	3
Semana 1 · El Dato y la Gestión de Datos.....	4
1.1 ¿Qué es un dato?.....	4
1.2 Dato, información y conocimiento: la jerarquía DIKW.....	4
1.3 Tipos de datos según su estructura.....	5
1.4 Ciclo de vida del dato.....	5
1.5 Calidad del dato.....	6
1.6 El dato como activo estratégico y la gestión de datos.....	7
1.7 Relación con Big Data y Ciencia de Datos.....	7
Semana 2 · SQLite Básico: Consultas y Filtros.....	9
2.1 Base de datos, tablas, filas y columnas.....	9
2.2 ¿Qué es SQL y para qué se utiliza?.....	9
2.3 ¿Qué es SQLite?.....	10
2.4 SELECT, FROM y LIMIT.....	10
2.5 DISTINCT.....	11
2.6 WHERE y operadores de comparación.....	11
2.7 ORDER BY.....	12
2.8 AND y OR.....	12
2.9 Funciones de agregación: COUNT, SUM, AVG, MAX y MIN.....	12
2.10 GROUP BY.....	13
2.11 Cómo interpretar un resultado.....	14
Semana 3 · SQL Avanzado: JOINS y Agrupaciones.....	15
3.1 ¿Por qué una base de datos utiliza varias tablas?.....	15
3.2 Clave primaria (PK) y clave foránea (FK).....	15
3.3 INNER JOIN.....	16
3.4 LEFT JOIN.....	16
3.5 JOIN de múltiples tablas.....	17
3.6 GROUP BY y HAVING.....	18
3.7 Subconsultas.....	19
3.8 Buenas prácticas de escritura SQL.....	20
3.9 De la necesidad de negocio a la consulta.....	20
Semana 4 · Modelado Relacional.....	21
4.1 El modelo relacional.....	21
4.2 Entidades y atributos.....	21
4.3 Claves primarias y foráneas en el diseño.....	21
4.4 Relaciones y cardinalidad.....	21
4.5 Normalización: 1FN, 2FN y 3FN.....	22
4.6 Del modelo a las tablas en SQLite.....	23
4.7 Errores frecuentes de diseño.....	24
4.8 Buen modelo, mejores datos.....	24
Integración de la Unidad 1.....	25
Mapa conceptual del recorrido.....	25
Caso integrador.....	25
Checklist de dominio de la Unidad 1.....	26
Conclusiones.....	26
Referencias bibliográficas.....	26

Introducción a la Unidad 1

Toda organización registra datos: una tienda anota cada venta, un banco cada transacción, una clínica cada atención. Sin embargo, registrar no es lo mismo que **gestionar**. Para que los datos sirvan para decidir, deben tener un significado claro, guardarse de forma ordenada, poder consultarse con precisión y mantener una calidad aceptable. Esta unidad construye esa base.

El recorrido es progresivo. Primero comprenderemos qué es un dato y por qué una organización debe cuidarlo (Semana 1). Luego aprenderemos a hacerle preguntas a una tabla con SQL (Semana 2). Después combinaremos información repartida en varias tablas (Semana 3) y, finalmente, entenderemos cómo se diseñan esas tablas para que los datos sean coherentes (Semana 4).



¿Qué aprenderemos?

Semana	Pregunta que responde	Al terminar, podrás...
1. El Dato y la Gestión de Datos	¿Qué es un dato y por qué una organización debe gestionarlo?	Clasificar los datos de una organización, describir su ciclo de vida y evaluar su calidad.
2. SQLite Básico	¿Cómo obtengo respuestas a partir de una tabla?	Escribir consultas SQL con filtros, orden y agregaciones, e interpretar sus resultados.
3. SQL Avanzado	¿Cómo combino información guardada en varias tablas?	Relacionar tablas con JOIN, filtrar grupos con HAVING y usar subconsultas para responder preguntas de negocio.
4. Modelado Relacional	¿Cómo deben diseñarse las tablas y sus relaciones?	Construir e interpretar un modelo relacional normalizado hasta la Tercera Forma Normal.

Cómo leer este material. Los conceptos principales se presentan con una misma secuencia: **concepto** (qué es), **contexto** (dónde aparece), **ejemplo** (un caso concreto), **aplicación** (para qué lo usa un profesional de datos), **interpretación** (qué significa el resultado) y **conexión** (cómo se enlaza con el resto de la unidad). Los ejemplos SQL siguen siempre el orden *pregunta de negocio* → *consulta* → *resultado esperado* → *interpretación*.

Caso transversal: RutaMarket

Para dar continuidad a las cuatro semanas trabajaremos con **RutaMarket S.A.C.**, una empresa peruana de comercio electrónico *ficticia, creada con fines académicos*. RutaMarket vende en línea productos de tecnología, hogar, deportes y libros a clientes de distintas ciudades del país. Su operación diaria genera datos sobre **clientes, productos, categorías, pedidos, detalle de pedidos y ventas**, además de reseñas, fotografías de productos y mensajes de atención al cliente.

La gerencia comercial necesita responder preguntas como: *¿qué productos generan más ingresos?*, *¿quiénes son nuestros mejores clientes?*, *¿hay clientes registrados que nunca compraron?*, *¿podemos confiar en las cifras?* Cada semana aportará una pieza para responderlas.

Semana	Cómo interviene el caso
1	Identificamos qué datos genera RutaMarket, cómo se clasifican, qué recorrido siguen y por qué su calidad importa.
2	Consultamos la tabla ventas (un registro por producto vendido en marzo de 2026) con SQL básico.
3	Relacionamos las tablas clientes , pedidos , detalle_pedido , productos y categorías mediante JOIN.
4	Comprendemos por qué esas tablas se diseñaron así y construimos el modelo relacional normalizado.

Sobre los datos del caso. Las tablas contienen pocas filas a propósito, para que puedas verificar los resultados a mano. Todas las consultas de este material fueron ejecutadas en SQLite sobre esos datos; los resultados mostrados son los que obtendrás al reproducirlas. Los montos están expresados en soles (S/).

Semana 1 · El Dato y la Gestión de Datos

Pregunta guía: ¿qué datos genera una organización y qué debe hacer con ellos para que sean útiles?

Antes de escribir una sola consulta es necesario entender con qué trabajamos. Un error frecuente al empezar en el mundo de los datos es pensar que el valor está en la herramienta. En realidad, la herramienta solo amplifica lo que ya existe: si los datos no tienen significado claro o están mal registrados, cualquier análisis posterior heredará esos problemas.

1.1 ¿Qué es un dato?

Concepto. Un **dato** es la representación registrada de un hecho, una observación o una medición: un número, un texto, una fecha, una imagen o un sonido. Por sí solo, un dato no explica nada; necesita **contexto** para adquirir significado.

Contexto. En RutaMarket, cada vez que un cliente se registra, agrega un producto al carrito, paga o escribe una reseña, se genera un dato. La empresa produce datos continuamente, aunque nadie los esté mirando.

Ejemplo. Observa estos valores aislados:

Dato aislado	¿Qué podría ser?	Con contexto en RutaMarket
240	¿Un precio? ¿Una cantidad? ¿Un código?	Monto en soles de la venta de 2 audífonos en el pedido 105
"Lima"	¿Una ciudad? ¿Un apellido? ¿Una sede?	Ciudad de residencia de la clienta Ana Torres
2026-03-15	¿Fecha de registro? ¿De entrega? ¿De pago?	Fecha en que se realizó el pedido 105

Aplicación. Lo primero que hace un profesional de datos ante un conjunto de datos nuevo es preguntar *qué representa cada valor*: su unidad, su origen y su fecha. Sin esa información, un análisis puede ser técnicamente correcto y aun así estar equivocado.

Conexión. En la Semana 2 veremos que una tabla le da contexto a cada dato: la **columna** indica qué es y la **fila** indica a qué entidad pertenece.

1.2 Dato, información y conocimiento: la jerarquía DIKW

Concepto. Los datos se transforman en valor de manera progresiva. La jerarquía **DIKW** (*Data, Information, Knowledge, Wisdom*), difundida por Ackoff (1989) y analizada por Rowley (2007), describe ese recorrido en cuatro niveles.

	Dato	Información	Conocimiento
¿Qué es?	Hecho registrado, sin interpretar	Datos organizados y puestos en contexto	Comprensión de patrones y causas
¿Qué pregunta responde?	¿Qué se registró?	¿Qué ocurrió? ¿Cuánto? ¿Dónde?	¿Por qué ocurre? ¿Qué suele pasar?
Ejemplo en RutaMarket	Pedido 105: 2 audífonos, S/ 240	En marzo, Tecnología generó casi la mitad de los ingresos	Los clientes de Lima concentran las compras de tecnología
¿Cómo se obtiene?	Captura (formulario, sistema de ventas)	Organización, consulta y resumen (SQL)	Análisis, comparación y experiencia



Figura 1. Jerarquía DIKW aplicada al caso RutaMarket.

Interpretación. El salto de **dato a información** requiere organizar y resumir; el salto de **información a conocimiento** requiere comparar, explicar y validar. La **sabiduría** corresponde al juicio para decidir, por ejemplo, priorizar el abastecimiento de tecnología para Lima. DIKW es un modelo conceptual: en la práctica los límites entre niveles no siempre son nítidos, pero ayuda a preguntarse en qué nivel se encuentra un análisis.

Conexión. Las consultas SQL de las Semanas 2 y 3 son precisamente la herramienta que convierte datos en información.

1.3 Tipos de datos según su estructura

Concepto. Según qué tan organizados estén, los datos se clasifican en **estructurados**, **semiestructurados** y **no estructurados**. La clasificación importa porque determina cómo se almacenan y con qué herramientas se pueden consultar.

	Estructurados	Semiestructurados	No estructurados
Organización	Filas y columnas con un esquema fijo	Etiquetas o claves que describen los datos, sin esquema rígido	Sin un esquema predefinido
Formatos típicos	Tablas de bases de datos relacionales, CSV	JSON, XML	Texto libre, imágenes, audio, video, PDF
Ejemplo en RutaMarket	Tablas de clientes, productos y pedidos	Confirmación de pago enviada por la pasarela en JSON	Reseñas, fotos de productos, chats de atención
¿Cómo se consulta?	Directamente con SQL	Se lee su estructura y se extraen los campos necesarios	Requiere procesamiento previo para extraer datos útiles

Ejemplo. La pasarela de pagos informa a RutaMarket del resultado de cada pago con un mensaje semiestructurado como este:

```
{ "id_pedido": 105, "estado_pago": "aprobado",
  "monto": 240.00, "moneda": "PEN", "metodo": "tarjeta" }
```

Interpretación. Cada valor está acompañado de su nombre ("**monto**": 240.00), por eso es fácil extraerlo y guardarlo en una tabla. Una reseña como "*Llegó rápido, pero la caja estaba dañada*" contiene información valiosa (satisfacción, problema logístico), pero no tiene campos: antes de analizarla hay que interpretarla y codificarla.

Aplicación. Muchas organizaciones generan grandes volúmenes de datos no estructurados. Su aprovechamiento suele comenzar convirtiéndolos en datos estructurados. Por esa razón, esta unidad se concentra en los datos estructurados y en las bases de datos relacionales: son el punto de llegada de buena parte del trabajo con datos.

1.4 Ciclo de vida del dato

Concepto. El **ciclo de vida del dato** describe las etapas por las que pasa un dato desde que se origina hasta que se archiva o elimina. Pensar en el ciclo completo ayuda a entender que la calidad y la utilidad de un dato dependen de todas las etapas, no solo del análisis.



Las decisiones generan nuevos datos (nuevas ventas, nuevos registros)

Figura 2. Ciclo de vida del dato.

Etapas	Qué ocurre	En RutaMarket
1. Captura	El dato se origina en formularios, transacciones, sensores o clics	El cliente completa su registro y realiza el pedido 105
2. Almacenamiento	Se guarda en una base de datos u otro repositorio	El pedido se registra en las tablas <code>pedidos</code> y <code>detalle_pedido</code>
3. Limpieza y preparación	Se detectan y corrigen errores, vacíos y duplicados	Se unifica "lima" y "Lima"; se elimina un cliente duplicado
4. Análisis	Se consultan y resumen los datos para encontrar patrones	Consulta SQL de ingresos por categoría
5. Visualización y comunicación	Los hallazgos se presentan en tablas, gráficos o reportes	Reporte mensual para la gerencia comercial
6. Decisión	La organización actúa con base en la evidencia	Aumentar el stock de tecnología para abril
7. Archivo o eliminación	El dato se conserva o se elimina según su utilidad y la normativa	Tratamiento de datos personales según la Ley N.º 29733

Interpretación. Un error introducido en la captura (una ciudad mal escrita) viaja por todo el ciclo y termina afectando la decisión. Además, las decisiones generan nuevos datos: la compra de más stock produce nuevas ventas que vuelven a iniciar el ciclo. En el Perú, el tratamiento de datos personales, incluida su conservación y eliminación, está regulado por la Ley N.º 29733, Ley de Protección de Datos Personales.

Conexión. Esta unidad trabaja sobre todo en las etapas de **almacenamiento** (Semana 4: cómo diseñar las tablas) y **análisis** (Semanas 2 y 3: cómo consultarlas).

1.5 Calidad del dato

Concepto. La **calidad del dato** es el grado en que los datos son adecuados para el uso que se les quiere dar (Wang y Strong, 1996). En el ámbito de datos se resume con el principio **GIGO** (*Garbage In, Garbage Out*): si entran datos deficientes, salen resultados deficientes, sin importar cuán sofisticada sea la herramienta. La calidad suele evaluarse en varias dimensiones (DAMA International, 2017):

Dimensión	¿Qué verifica?	Problema de ejemplo en RutaMarket
Exactitud	El dato refleja correctamente la realidad	La ciudad registrada es "Arequpa" en lugar de "Arequipa"
Compleitud	No faltan valores necesarios	Clientes sin ciudad registrada
Consistencia	El mismo dato no se contradice entre registros o sistemas	"Lima" y "lima"; la ciudad del cliente difiere entre dos sistemas
Oportunidad	El dato está disponible cuando se necesita	Las ventas de marzo se consolidan recién a fines de abril
Unicidad	Cada entidad está registrada una sola vez	La misma cliente aparece dos veces con distinto código
Validez	El dato cumple el formato y las reglas definidas	Fecha "2026-02-30" o correo sin dominio completo

Ejemplo. Supón que el área comercial exporta este fragmento de la lista de clientes:

id_cliente	nombre	ciudad	email	fecha_registro
1	Ana Torres	Lima	ana.torres@correo.pe	2025-11-04
7	Ana Torres	lima	ana.torres@correo.pe	04/11/2025
8	Pedro Salas		pedro.salas@correo	2026-02-30
9	Lucía Vega	Arequipa	lucia.vega@correo.pe	2026-03-01

Interpretación. Si con esta tabla contamos clientes por ciudad, obtendremos grupos distintos para "Lima" y "lima", Ana Torres se contará dos veces (problema de **unicidad**), Pedro Salas quedará sin ciudad (**completitud**) y su fecha no existe (**validez**). Ninguna consulta, por correcta que sea, puede compensar esos defectos: el resultado será un número preciso, pero falso.

Aplicación. Antes de analizar, el profesional de datos revisa la calidad y documenta los problemas encontrados. Detectarlos a tiempo es mucho más económico que corregir una decisión tomada con cifras erróneas.

Conexión. En la Semana 2 verás que SQL distingue "Lima" de "lima"; en la Semana 4, que un buen diseño (claves, restricciones) impide muchos de estos errores desde el origen.

1.6 El dato como activo estratégico y la gestión de datos

Concepto. Un **activo** es un recurso que genera valor para la organización. Los datos cumplen esa condición: permiten conocer a los clientes, anticipar la demanda, detectar riesgos y evaluar resultados. A diferencia de un activo físico, un dato no se agota al usarse y puede reutilizarse en muchos análisis; pero pierde valor si es inexacto, está desactualizado o nadie puede encontrarlo.

Contexto. El uso de datos para decidir está presente en casi todos los sectores:

Sector	Datos que registra	Uso para la toma de decisiones
Comercio minorista y electrónico	Ventas, productos, clientes, navegación	Planificar inventario, recomendar productos, definir promociones
Banca y finanzas	Transacciones, historial crediticio	Evaluar créditos, detectar operaciones inusuales
Salud	Atenciones, diagnósticos, resultados de laboratorio	Planificar recursos, hacer seguimiento a pacientes
Transporte y logística	Rutas, tiempos de entrega, incidencias	Optimizar rutas, estimar tiempos de despacho

Concepto. La **gestión de datos** es el conjunto de planes, políticas, prácticas y responsabilidades que permiten que los datos de una organización se obtengan, protejan, mantengan y aprovechen como activos (DAMA International, 2017).

Sin gestión de datos	Con gestión de datos
Cada área mantiene su propia hoja de cálculo de ventas	Existe una fuente oficial de ventas, accesible para quien la necesita
"Ventas del mes" significa cosas distintas para Comercial y Finanzas	Las métricas tienen una definición común y documentada
Los errores se descubren cuando el reporte ya fue presentado	Hay controles de calidad y responsables de cada conjunto de datos
Cualquier persona puede ver datos personales de los clientes	El acceso se asigna según roles y se cumple la normativa

Aplicación. Para RutaMarket, gestionar sus datos significa, por ejemplo, definir quién es responsable de la tabla de clientes, qué formato debe tener cada columna y qué significa exactamente "venta". Sin esos acuerdos, dos analistas pueden reportar cifras distintas para la misma pregunta.

1.7 Relación con Big Data y Ciencia de Datos

Big Data se refiere a conjuntos de datos cuyo **volumen**, **velocidad** de generación y **variedad** de formatos superan la capacidad de las herramientas tradicionales; estas tres características fueron planteadas por Laney (2001). La **Ciencia de Datos** reúne principios y métodos para extraer conocimiento útil a partir de datos con el fin de apoyar decisiones (Provost y Fawcett, 2013). Ambas disciplinas se apoyan en los fundamentos de esta semana:

Concepto de la Semana 1	Por qué es indispensable en Big Data y Ciencia de Datos
Dato e información (DIKW)	Todo proyecto busca subir en la jerarquía: pasar de registros a conocimiento que oriente decisiones.
Tipos de datos	La variedad de Big Data es precisamente la mezcla de datos estructurados, semiestructurados y no estructurados.
Ciclo de vida	Organiza el trabajo: ningún análisis empieza en el análisis; empieza en la captura y el almacenamiento.
Calidad del dato	Con grandes volúmenes, un error sistemático se repite miles de veces; la calidad deficiente invalida las conclusiones.
Gestión de datos	Garantiza que los datos se encuentren, se entiendan y se usen de forma segura y consistente.

Cierre de la Semana 1

Lo que debes dominar

- Distinguir dato, información y conocimiento, y ubicar un ejemplo en la jerarquía DIKW.
- Clasificar datos como estructurados, semiestructurados o no estructurados y reconocer cómo se consulta cada tipo.
- Describir las etapas del ciclo de vida del dato con un ejemplo de una organización.
- Identificar problemas de calidad usando las seis dimensiones (exactitud, completitud, consistencia, oportunidad, unicidad, validez).
- Explicar por qué el dato es un activo estratégico y qué aporta la gestión de datos a una organización.

Comprueba tu aprendizaje

1. ¿Por qué el valor "240" es un dato y no información? ¿Qué le falta para convertirse en información?
2. Clasifica y justifica: una factura electrónica en XML, la foto de un producto, la tabla de pedidos y la grabación de una llamada de atención.
3. Una tabla tiene 1 000 clientes, pero 80 están repetidos con distinto código. ¿Qué dimensión de calidad se afecta y qué ocurriría con el indicador "número de clientes"?
4. El área de Finanzas reporta S/ 50 000 en ventas de marzo y el área Comercial, S/ 58 000. Propón dos posibles causas relacionadas con la gestión de datos.

Conexión con la Semana 2

Semana 1 → ya comprendimos qué son los datos, cómo se clasifican y por qué deben gestionarse con calidad.

Semana 2 → ahora aprenderemos a **consultarlos**: guardaremos las ventas de RutaMarket en una tabla y le haremos preguntas con SQL. Pasaremos, en la práctica, del nivel *dato* al nivel *información* de la jerarquía DIKW.

Semana 2 · SQLite Básico: Consultas y Filtros

Pregunta guía: ¿cómo obtengo respuestas precisas a partir de los datos guardados en una tabla?

En la Semana 1 vimos que los datos adquieren significado cuando se organizan. Ahora daremos el primer paso práctico: guardar los datos en una tabla y consultarlos con **SQL**, el lenguaje estándar de las bases de datos relacionales.

2.1 Base de datos, tablas, filas y columnas

Concepto. Una **tabla** organiza datos del mismo tipo en **filas** y **columnas**. Cada **columna** representa una característica (qué es el dato) y cada **fila** representa un caso concreto (a quién o a qué pertenece). Una **base de datos** es un conjunto organizado de tablas relacionadas, administrado por un **sistema gestor de bases de datos** (SGBD).

Término	Sinónimos frecuentes	¿Qué representa?
Tabla	Relación, entidad (en diseño), dataset	Conjunto de registros del mismo tipo
Fila	Registro, tupla, observación	Un caso concreto: una venta, un cliente, un producto
Columna	Campo, atributo, variable	Una característica registrada para todos los casos
Celda	Valor	El dato específico en la intersección de una fila y una columna
Clave primaria	Primary Key (PK), identificador	Columna cuyo valor identifica cada fila sin repetirse

Ejemplo. RutaMarket exporta sus ventas de marzo de 2026 en la tabla **ventas**. Cada fila corresponde a un **producto vendido dentro de un pedido**:

id_venta	fecha	cliente	ciudad	producto	categoria	cantidad	precio_unitario	total
1	2026-03-02	Ana Torres	Lima	Audífonos inalámbricos	Tecnología	1	120.00	120.00
2	2026-03-02	Ana Torres	Lima	Mouse inalámbrico	Tecnología	2	45.00	90.00
3	2026-03-05	Luis Quispe	Arequipa	Licuada 600 W	Hogar	1	189.90	189.90
4	2026-03-09	Ana Torres	Lima	Teclado mecánico	Tecnología	1	210.00	210.00
5	2026-03-09	Ana Torres	Lima	Mat de yoga	Deportes	1	55.00	55.00
6	2026-03-12	María Huamán	Cusco	Mancuernas 5 kg	Deportes	2	80.00	160.00
7	2026-03-12	María Huamán	Cusco	Mat de yoga	Deportes	1	55.00	55.00
8	2026-03-15	Jorge Ramos	Lima	Audífonos inalámbricos	Tecnología	2	120.00	240.00
9	2026-03-20	Rosa Flores	Trujillo	Juego de sábanas	Hogar	2	95.00	190.00
10	2026-03-24	Luis Quispe	Arequipa	Mouse inalámbrico	Tecnología	1	45.00	45.00
11	2026-03-24	Luis Quispe	Arequipa	Teclado mecánico	Tecnología	1	210.00	210.00
12	2026-03-28	Jorge Ramos	Lima	Licuada 600 W	Hogar	1	189.90	189.90
13	2026-03-28	Jorge Ramos	Lima	Mancuernas 5 kg	Deportes	1	80.00	80.00

Interpretación. La tabla tiene 13 filas y 9 columnas. El valor "Lima" de la fila 1 significa "ciudad de la cliente que realizó esa venta". Observa también que "Ana Torres, Lima" se repite en cuatro filas: guarda esta observación, porque será clave en las Semanas 3 y 4.

2.2 ¿Qué es SQL y para qué se utiliza?

Concepto. **SQL** (*Structured Query Language*) es el lenguaje estándar para definir, consultar y modificar datos en bases de datos relacionales. Surgió en IBM en la década de 1970 (Chamberlin y Boyce, 1974) y hoy es un estándar internacional. Es un lenguaje **declarativo**: se indica *qué* resultado se quiere, no *cómo* obtenerlo; el gestor decide la forma de calcularlo.

Contexto. SQL se utiliza en sistemas de ventas, bancos, universidades y hospitales, y está detrás de muchos reportes y tableros de gestión. Analistas, ingenieros y científicos de datos lo usan para extraer y resumir la información que necesitan.

Aplicación. Cada consulta es una pregunta estructurada. Sus cláusulas responden a preguntas simples:

Cláusula	Pregunta que responde	Orden lógico de ejecución
SELECT	¿Qué columnas o cálculos quiero ver?	5
FROM	¿De qué tabla?	1
WHERE	¿Qué filas cumplen la condición?	2
GROUP BY	¿Cómo agrupo las filas?	3
HAVING	¿Qué grupos conservo? (Semana 3)	4
ORDER BY	¿En qué orden muestro el resultado?	6
LIMIT	¿Cuántas filas muestro como máximo?	7

Interpretación. Las cláusulas se **escriben** en el orden de la tabla, pero el gestor las **procesa** en otro orden: primero lee la tabla (FROM), luego filtra filas (WHERE), agrupa (GROUP BY) y recién después calcula lo que se muestra (SELECT). Entender este orden evita errores frecuentes, como los que analizaremos con WHERE y HAVING en la Semana 3.

2.3 ¿Qué es SQLite?

Concepto. **SQLite** es un gestor de bases de datos relacionales que no requiere un servidor: toda la base de datos se guarda en un único archivo (por ejemplo, [rutamarket.db](#)). Es gratuito y de dominio público, y está incorporado en numerosas aplicaciones móviles y de escritorio.

Característica	SQLite	Gestores cliente-servidor (MySQL, PostgreSQL)
Instalación	Mínima: la base es un archivo	Requiere instalar y configurar un servidor
Usuarios simultáneos	Adecuado para uso individual o de baja concurrencia	Diseñados para muchos usuarios a la vez
Uso típico	Aprendizaje, prototipos, aplicaciones locales y móviles	Sistemas empresariales de gran escala

Aplicación. Para practicar puedes usar una herramienta gráfica gratuita como **DB Browser for SQLite** o la consola [sqlite3](#). Lo aprendido en SQLite se transfiere casi sin cambios a otros gestores. SQLite trabaja con pocos tipos de almacenamiento: **INTEGER** (enteros), **REAL** (decimales), **TEXT** (texto y fechas en formato AAAA-MM-DD) y **NULL** (ausencia de valor).

2.4 SELECT, FROM y LIMIT

SELECT indica qué columnas mostrar, **FROM** de qué tabla y **LIMIT** cuántas filas como máximo. Es la primera consulta que se ejecuta ante una tabla nueva: permite **explorar** su contenido sin traer miles de filas.

PREGUNTA DE NEGOCIO

¿Qué aspecto tienen los registros de la tabla de ventas?

CONSULTA SQL

```
SELECT fecha, cliente, producto, total
FROM ventas
LIMIT 5;
```

RESULTADO ESPERADO

fecha	cliente	producto	total
2026-03-02	Ana Torres	Audífonos inalámbricos	120.00
2026-03-02	Ana Torres	Mouse inalámbrico	90.00
2026-03-05	Luis Quispe	Licuada 600 W	189.90
2026-03-09	Ana Torres	Teclado mecánico	210.00
2026-03-09	Ana Torres	Mat de yoga	55.00

INTERPRETACIÓN

Vemos las cinco primeras ventas con cuatro de sus nueve columnas. Esta exploración inicial permite confirmar qué contiene cada columna y en qué formato está (por ejemplo, que la fecha usa AAAA-MM-DD). Importante: sin **ORDER BY**, la base de datos no garantiza qué filas serán "las primeras"; **LIMIT** sirve para explorar, no para identificar las ventas más recientes o más altas.

2.5 DISTINCT

DISTINCT elimina las filas repetidas del resultado. Se usa para conocer los **valores diferentes** que toma una columna.

PREGUNTA DE NEGOCIO

¿En qué categorías se registraron ventas en marzo?

CONSULTA SQL

```
SELECT DISTINCT categoria
FROM ventas;
```

RESULTADO ESPERADO

categoria
Tecnología
Hogar
Deportes

INTERPRETACIÓN

Aunque la tabla tiene 13 filas, solo hay tres categorías con ventas. Si el catálogo de RutaMarket tiene cuatro categorías, la cuarta (Libros) no vendió nada en marzo. Este hallazgo no puede verse en la tabla `ventas`, porque en ella solo aparece lo que se vendió; en la Semana 3 aprenderemos a detectarlo cruzando tablas.

2.6 WHERE y operadores de comparación

WHERE filtra las filas que cumplen una condición. La condición se construye con operadores de comparación:

Operador	Significado	Ejemplo
=	Igual a	ciudad = 'Lima'
<> o !=	Distinto de	categoria <> 'Hogar'
> / <	Mayor que / menor que	total > 150
>= / <=	Mayor o igual / menor o igual	cantidad >= 2
BETWEEN	Dentro de un rango (incluye extremos)	fecha BETWEEN '2026-03-01' AND '2026-03-15'
IN	Coincide con algún valor de una lista	ciudad IN ('Cusco', 'Trujillo')

PREGUNTA DE NEGOCIO

¿Qué ventas se hicieron a clientes de Lima?

CONSULTA SQL

```
SELECT fecha, cliente, producto, total
FROM ventas
WHERE ciudad = 'Lima';
```

RESULTADO ESPERADO

fecha	cliente	producto	total
2026-03-02	Ana Torres	Audífonos inalámbricos	120.00
2026-03-02	Ana Torres	Mouse inalámbrico	90.00
2026-03-09	Ana Torres	Teclado mecánico	210.00
2026-03-09	Ana Torres	Mat de yoga	55.00
2026-03-15	Jorge Ramos	Audífonos inalámbricos	240.00
2026-03-28	Jorge Ramos	Licuada 600 W	189.90
2026-03-28	Jorge Ramos	Mancuernas 5 kg	80.00

INTERPRETACIÓN

Siete de las 13 ventas corresponden a clientes de Lima, concentradas en solo dos personas (Ana Torres y Jorge Ramos). Para la gerencia, esto sugiere que Lima es el mercado principal, pero también que depende de pocos clientes.

Dos detalles prácticos: el texto se escribe entre comillas simples y, en SQLite, = distingue mayúsculas de minúsculas. Si algún registro dijera "lima", no aparecería en el resultado. Es el problema de **consistencia** visto en la Semana 1, ahora con efectos concretos.

2.7 ORDER BY

ORDER BY ordena el resultado según una o más columnas: **ASC** (ascendente, valor por defecto) o **DESC** (descendente).

PREGUNTA DE NEGOCIO

¿Cuáles fueron las ventas de S/ 150 o más, de mayor a menor?

CONSULTA SQL

```
SELECT fecha, cliente, producto, total
FROM ventas
WHERE total >= 150
ORDER BY total DESC;
```

RESULTADO ESPERADO

fecha	cliente	producto	total
2026-03-15	Jorge Ramos	Audífonos inalámbricos	240.00
2026-03-09	Ana Torres	Teclado mecánico	210.00
2026-03-24	Luis Quispe	Teclado mecánico	210.00
2026-03-20	Rosa Flores	Juego de sábanas	190.00
2026-03-05	Luis Quispe	Licuada 600 W	189.90
2026-03-28	Jorge Ramos	Licuada 600 W	189.90
2026-03-12	María Huamán	Mancuernas 5 kg	160.00

INTERPRETACIÓN

Siete ventas alcanzan o superan S/ 150. La mayor es la de audífonos a Jorge Ramos (S/ 240), seguida por dos ventas de teclados. Ordenar permite priorizar: si el área de logística quiere revisar el embalaje de los envíos de mayor valor, esta lista le indica por dónde empezar.

2.8 AND y OR

AND exige que se cumplan **todas** las condiciones; **OR** exige que se cumpla **al menos una**. Cuando se combinan, **AND** se evalúa antes que **OR**, por lo que conviene usar paréntesis para dejar clara la intención.

PREGUNTA DE NEGOCIO

¿Qué ventas de Tecnología se hicieron a clientes de Lima?

CONSULTA SQL

```
SELECT cliente, ciudad, producto, total
FROM ventas
WHERE categoria = 'Tecnología'
AND ciudad = 'Lima';
```

RESULTADO ESPERADO

cliente	ciudad	producto	total
Ana Torres	Lima	Audífonos inalámbricos	120.00
Ana Torres	Lima	Mouse inalámbrico	90.00
Ana Torres	Lima	Teclado mecánico	210.00
Jorge Ramos	Lima	Audífonos inalámbricos	240.00

INTERPRETACIÓN

Cuatro ventas cumplen ambas condiciones a la vez. Suman S/ 660, lo que indica que la tecnología explica buena parte de lo que se vende en Lima: un dato útil para planificar el stock de esa ciudad.

Con **OR** basta que se cumpla una condición: **WHERE ciudad = 'Cusco' OR ciudad = 'Trujillo'** devuelve las tres ventas hechas a clientes de esas ciudades (dos de María Huamán y una de Rosa Flores). La misma condición se escribe de forma más compacta con **WHERE ciudad IN ('Cusco', 'Trujillo')**.

2.9 Funciones de agregación: COUNT, SUM, AVG, MAX y MIN

Las funciones de agregación resumen muchas filas en un solo valor. Son el paso decisivo de *dato* a *información*.

Función	Calcula	Ejemplo de pregunta
COUNT	Número de filas (o de valores no nulos de una columna)	¿Cuántas ventas hubo?
SUM	Suma de los valores	¿Cuánto se facturó?
AVG	Promedio	¿Cuál es el monto medio por venta?
MAX / MIN	Valor máximo / mínimo	¿Cuál fue la venta más alta y la más baja?

PREGUNTA DE NEGOCIO

¿Cuál es el resumen general de las ventas de marzo?

CONSULTA SQL

```
SELECT COUNT(*)      AS lineas_venta,
       SUM(total)     AS ingresos,
       ROUND(AVG(total), 2) AS ticket_promedio,
       MAX(total)     AS venta_maxima,
       MIN(total)     AS venta_minima
FROM ventas;
```

RESULTADO ESPERADO

lineas_venta	ingresos	ticket_promedio	venta_maxima	venta_minima
13	1,834.80	141.14	240.00	45.00

INTERPRETACIÓN

En marzo hubo 13 líneas de venta por S/ 1,834.80. El promedio de S/ 141.14 corresponde a cada **línea de venta** (un producto dentro de un pedido), no a cada pedido. Como las 13 líneas pertenecen a 8 pedidos, el monto promedio por pedido es S/ 1,834.80 / 8 = S/ 229.35.

Esta diferencia no es un detalle técnico: si dos analistas usan definiciones distintas de "ticket promedio", reportarán cifras distintas. Por eso la gestión de datos exige definir las métricas (Semana 1). **AS** asigna un nombre legible a cada columna calculada; **ROUND(valor, 2)** redondea a dos decimales.

2.10 GROUP BY

GROUP BY forma grupos de filas que comparten un valor y aplica la función de agregación **a cada grupo**. Toda columna del **SELECT** que no esté dentro de una función de agregación debe figurar en el **GROUP BY**.

PREGUNTA DE NEGOCIO

¿Cuánto vendió cada categoría?

CONSULTA SQL

```
SELECT categoria,
       COUNT(*) AS lineas_venta,
       SUM(total) AS ingresos
FROM ventas
GROUP BY categoria
ORDER BY ingresos DESC;
```

RESULTADO ESPERADO

categoria	lineas_venta	ingresos
Tecnología	6	915.00
Hogar	3	569.80
Deportes	4	350.00

INTERPRETACIÓN

Tecnología generó S/ 915.00, casi la mitad del total (S/ 1,834.80). Hogar tiene menos ventas que Deportes (3 frente a 4), pero más ingresos, porque sus productos tienen mayor precio. Contar ventas y sumar ingresos responden preguntas distintas; el tomador de decisiones debe saber cuál necesita.

PREGUNTA DE NEGOCIO

¿Cuáles son los cinco productos con mayores ventas?

CONSULTA SQL

```
SELECT producto,
       SUM(cantidad) AS unidades,
       SUM(total) AS ingresos
FROM ventas
GROUP BY producto
ORDER BY ingresos DESC
LIMIT 5;
```

RESULTADO ESPERADO

producto	unidades	ingresos
Teclado mecánico	2	420.00
Licadora 600 W	2	379.80
Audífonos inalámbricos	3	360.00
Mancuernas 5 kg	3	240.00
Juego de sábanas	2	190.00

INTERPRETACIÓN

El teclado mecánico lidera en ingresos (S/ 420.00) aunque vendió solo 2 unidades, mientras que los audífonos vendieron más unidades (3) pero ingresaron menos. "Mayores ventas" puede significar más ingresos o más unidades: la consulta debe reflejar la pregunta real. Además, se trata de un solo mes y de pocos pedidos; antes de decidir, conviene confirmar el patrón con más periodos.

2.11 Cómo interpretar un resultado

Un resultado SQL siempre es "correcto" para la consulta que se escribió, pero puede no responder la pregunta que se tenía. Antes de comunicar una cifra, revisa:

Revisa	Pregunta de control
Grano	¿Qué representa cada fila del resultado: una venta, un pedido, una categoría?
Alcance y unidad	¿Qué filtros y periodo se aplicaron? ¿Son soles, unidades o número de registros?
Plausibilidad	¿El valor es razonable para el negocio? Un total negativo o un promedio desproporcionado indican un error.
Calidad de origen	¿Podría haber duplicados, vacíos o inconsistencias que distorsionen la cifra?

Cierre de la Semana 2

Lo que debes dominar

- Reconocer la estructura de una tabla (filas, columnas, celdas) y lo que representa cada fila.
- Explorar una tabla con **SELECT**, **FROM**, **LIMIT** y **DISTINCT**.
- Filtrar filas con **WHERE**, operadores de comparación, **AND**, **OR** e **IN**, y ordenar con **ORDER BY**.
- Resumir datos con **COUNT**, **SUM**, **AVG**, **MAX**, **MIN** y agruparlos con **GROUP BY**.
- Interpretar un resultado considerando su grano, alcance, unidad y calidad de origen.

Comprueba tu aprendizaje

1. ¿Por qué **SELECT * FROM ventas LIMIT 5** no sirve para conocer las cinco ventas más altas? ¿Cómo la corregirías?
2. Escribe una consulta que muestre cuántas unidades se vendieron por ciudad, de mayor a menor.
3. Un reporte indica "ticket promedio: S/ 141.14" y otro "S/ 229.35". ¿Pueden ambos ser correctos? Explica.
4. La consulta **WHERE ciudad = 'Lima' OR ciudad = 'Cusco' AND total > 150** devuelve más filas de las esperadas. ¿Por qué? ¿Cómo la corregirías?

Conexión con la Semana 3

Semana 2 → ya sabemos consultar, filtrar y resumir los datos de una tabla.

Semana 3 → pero la tabla **ventas** tiene límites: repite los datos de cada cliente en varias filas y no puede mostrar a los clientes que nunca compraron ni los productos que nunca se vendieron. Aprenderemos a trabajar con **varias tablas relacionadas** y a combinarlas con **JOIN**.

Semana 3 · SQL Avanzado: JOINS y Agrupaciones

Pregunta guía: ¿cómo respondo preguntas que requieren información guardada en distintas tablas?

3.1 ¿Por qué una base de datos utiliza varias tablas?

Contexto. La tabla `ventas` de la Semana 2 es cómoda para consultar, pero presenta tres problemas si se usa como único registro:

- **Repetición:** el nombre y la ciudad de Ana Torres se repiten en cuatro filas; la categoría de cada producto se repite en cada venta.
- **Riesgo de inconsistencia:** si Ana se muda a Arequipa, habría que actualizar cuatro filas; si se omite una, la base tendrá dos ciudades para la misma persona.
- **Información que no se puede guardar:** un cliente que se registró pero aún no compra, o un producto que todavía no se vendió, no tienen dónde registrarse.

Concepto. La solución es separar los datos en tablas, una por cada tipo de entidad, y conectarlas mediante columnas comunes. Así, cada dato se guarda **una sola vez**. En la base de datos de RutaMarket existen estas tablas:

Tabla	Qué guarda (una fila por...)	Columnas
<code>clientes</code>	cada cliente registrado	<code>id_cliente</code> , <code>nombre</code> , <code>ciudad</code> , <code>email</code> , <code>fecha_registro</code>
<code>categorias</code>	cada categoría del catálogo	<code>id_categoria</code> , <code>nombre_categoria</code>
<code>productos</code>	cada producto del catálogo	<code>id_producto</code> , <code>nombre_producto</code> , <code>id_categoria</code> , <code>precio_unitario</code>
<code>pedidos</code>	cada compra realizada	<code>id_pedido</code> , <code>id_cliente</code> , <code>fecha_pedido</code> , <code>estado</code>
<code>detalle_pedido</code>	cada producto dentro de un pedido	<code>id_pedido</code> , <code>id_producto</code> , <code>cantidad</code> , <code>precio_unitario</code>

Contenido de las tablas principales (los montos de `detalle_pedido` coinciden con la tabla `ventas` de la Semana 2):

<code>id_cliente</code>	<code>nombre</code>	<code>ciudad</code>	<code>fecha_registro</code>
1	Ana Torres	Lima	2025-11-04
2	Luis Quispe	Arequipa	2025-12-10
3	María Huamán	Cusco	2026-01-15
4	Jorge Ramos	Lima	2026-01-20
5	Rosa Flores	Trujillo	2026-02-02
6	Carlos Mendoza	Piura	2026-02-25

<code>id_producto</code>	<code>nombre_producto</code>	<code>id_categoria</code>	<code>precio_unitario</code>
1	Audífonos inalámbricos	1	120.00
2	Mouse inalámbrico	1	45.00
3	Teclado mecánico	1	210.00
4	Licuada 600 W	2	189.90
5	Juego de sábanas	2	95.00
6	Mancuernas 5 kg	3	80.00
7	Mat de yoga	3	55.00
8	Libro Introducción a SQL	4	65.00

`categorias`: 1 = Tecnología, 2 = Hogar, 3 = Deportes, 4 = Libros. `pedidos` tiene 8 filas (101 a 108) y `detalle_pedido`, 13; por ejemplo, (101, 1, 1, 120.00) indica que el pedido 101 incluyó 1 unidad del producto 1 a S/ 120.00.

Conexión. Separar tablas resuelve la repetición, pero crea una nueva necesidad: **volver a unirlos** cuando una pregunta lo requiere. Para eso existen las claves y los JOIN.

3.2 Clave primaria (PK) y clave foránea (FK)

Concepto. La **clave primaria** identifica de forma única cada fila de una tabla. La **clave foránea** es una columna que guarda el valor de la clave primaria de otra tabla y, con ello, establece la relación entre ambas.

	Clave primaria (PK)	Clave foránea (FK)
Función	Identificar cada fila sin ambigüedad	Referenciar una fila de otra tabla
¿Puede repetirse?	No, es única en su tabla	Sí: un cliente puede figurar en muchos pedidos
¿Puede quedar vacía?	No	Depende de la regla de negocio
Ejemplo	<code>id_cliente</code> en <code>clientes</code>	<code>id_cliente</code> en <code>pedidos</code>
Qué garantiza	Unicidad: no hay clientes duplicados con el mismo código	Integridad: no hay pedidos de clientes inexistentes

Interpretación. El pedido 105 tiene `id_cliente` = 4. Para saber quién lo hizo, se busca la fila 4 en `clientes`: Jorge Ramos, de Lima. Un JOIN hace exactamente esa búsqueda, para todas las filas a la vez.

3.3 INNER JOIN

INNER JOIN combina filas de dos tablas cuando cumplen la condición indicada en **ON** (normalmente, que la FK coincida con la PK). Solo devuelve las filas que tienen pareja en **ambas** tablas. Los **alias** (`p`, `c`) abrevian el nombre de cada tabla.

PREGUNTA DE NEGOCIO

¿Quién realizó cada pedido y desde qué ciudad?

CONSULTA SQL

```
SELECT p.id_pedido,
       c.nombre AS cliente,
       c.ciudad,
       p.fecha_pedido,
       p.estado
FROM pedidos AS p
INNER JOIN clientes AS c
      ON p.id_cliente = c.id_cliente
ORDER BY p.fecha_pedido;
```

RESULTADO ESPERADO

id_pedido	cliente	ciudad	fecha_pedido	estado
101	Ana Torres	Lima	2026-03-02	Entregado
102	Luis Quispe	Arequipa	2026-03-05	Entregado
103	Ana Torres	Lima	2026-03-09	Entregado
104	María Huamán	Cusco	2026-03-12	Entregado
105	Jorge Ramos	Lima	2026-03-15	Entregado
106	Rosa Flores	Trujillo	2026-03-20	Entregado
107	Luis Quispe	Arequipa	2026-03-24	En camino
108	Jorge Ramos	Lima	2026-03-28	En camino

INTERPRETACIÓN

Cada pedido aparece ahora con el nombre y la ciudad de su cliente, aunque esos datos están guardados en otra tabla. Se observa que Ana, Luis y Jorge hicieron dos pedidos cada uno (clientes recurrentes) y que los pedidos 107 y 108 aún están en camino. Carlos Mendoza no aparece: no tiene pedidos y, por lo tanto, no tiene pareja en `pedidos`.

3.4 LEFT JOIN

LEFT JOIN devuelve **todas** las filas de la tabla de la izquierda (la que está en **FROM**), tengan o no pareja en la tabla de la derecha. Cuando no hay pareja, las columnas de la derecha se completan con **NULL**.

PREGUNTA DE NEGOCIO

¿Qué pedidos tiene cada cliente registrado, incluidos quienes no han comprado?

CONSULTA SQL

```
SELECT c.nombre AS cliente,
       c.ciudad,
       p.id_pedido
FROM clientes AS c
LEFT JOIN pedidos AS p
      ON c.id_cliente = p.id_cliente
ORDER BY c.id_cliente;
```

RESULTADO ESPERADO

cliente	ciudad	id_pedido
Ana Torres	Lima	101
Ana Torres	Lima	103
Luis Quispe	Arequipa	102
Luis Quispe	Arequipa	107
María Huamán	Cusco	104
Jorge Ramos	Lima	105
Jorge Ramos	Lima	108
Rosa Flores	Trujillo	106
Carlos Mendoza	Piura	NULL

INTERPRETACIÓN

Aparecen los seis clientes. Carlos Mendoza figura con `id_pedido` en NULL: está registrado, pero no ha comprado. Con `INNER JOIN` habría desaparecido del resultado y el hallazgo pasaría inadvertido.

PREGUNTA DE NEGOCIO

¿Qué clientes registrados nunca han realizado un pedido?

CONSULTA SQL

```
SELECT c.nombre AS cliente, c.ciudad, c.fecha_registro
FROM clientes AS c
LEFT JOIN pedidos AS p
      ON c.id_cliente = p.id_cliente
WHERE p.id_pedido IS NULL;
```

RESULTADO ESPERADO

cliente	ciudad	fecha_registro
Carlos Mendoza	Piura	2026-02-25

INTERPRETACIÓN

Filtrar las filas sin pareja (`IS NULL`) convierte el `LEFT JOIN` en una lista de clientes inactivos. Carlos se registró el 25 de febrero y no compró en marzo. Para el área de marketing esto sugiere una acción concreta, como una comunicación de bienvenida o un cupón para la primera compra. Nota: para buscar valores vacíos se usa `IS NULL`, nunca `= NULL`.

	INNER JOIN	LEFT JOIN
Qué devuelve	Solo filas con coincidencia en ambas tablas	Todas las filas de la tabla izquierda, con o sin coincidencia
Filas sin pareja	Se descartan	Se conservan con NULL en las columnas de la derecha
Pregunta típica	¿Qué pedidos hizo cada cliente?	¿Qué clientes no han comprado? ¿Qué productos no se venden?

3.5 JOIN de múltiples tablas

Una pregunta puede requerir información de tres o más tablas. Se encadenan varios JOIN, siguiendo el camino de las claves: `detalle_pedido` → `productos` (por `id_producto`) → `categorias` (por `id_categoria`).

PREGUNTA DE NEGOCIO

¿Cuántas unidades e ingresos generó cada categoría?

CONSULTA SQL

```
SELECT ca.nombre_categoria AS categoria,
       SUM(d.cantidad) AS unidades,
       SUM(d.cantidad * d.precio_unitario) AS ingresos
FROM detalle_pedido AS d
INNER JOIN productos AS pr ON d.id_producto = pr.id_producto
INNER JOIN categorias AS ca ON pr.id_categoria = ca.id_categoria
GROUP BY ca.nombre_categoria
ORDER BY ingresos DESC;
```

RESULTADO ESPERADO

categoria	unidades	ingresos
Tecnología	8	915.00
Hogar	4	569.80
Deportes	5	350.00

INTERPRETACIÓN

Los ingresos por categoría coinciden exactamente con los obtenidos en la Semana 2 desde la tabla [ventas](#). No es casualidad: aquella tabla era una exportación construida a partir de estas cinco tablas. La diferencia es que ahora cada dato está guardado una sola vez y el resultado se calcula combinándolas.

PREGUNTA DE NEGOCIO

¿Cuántos pedidos hizo cada cliente y cuánto gastó en total?

CONSULTA SQL

```
SELECT c.nombre AS cliente,
       COUNT(DISTINCT p.id_pedido) AS pedidos,
       SUM(d.cantidad * d.precio_unitario) AS monto_total
FROM clientes AS c
INNER JOIN pedidos AS p ON c.id_cliente = p.id_cliente
INNER JOIN detalle_pedido AS d ON p.id_pedido = d.id_pedido
GROUP BY c.nombre
ORDER BY monto_total DESC;
```

RESULTADO ESPERADO

cliente	pedidos	monto_total
Jorge Ramos	2	509.90
Ana Torres	2	475.00
Luis Quispe	2	444.90
María Huamán	1	215.00
Rosa Flores	1	190.00

INTERPRETACIÓN

Jorge Ramos es el cliente con mayor monto acumulado (S/ 509.90), seguido de cerca por Ana Torres y Luis Quispe. `COUNT(DISTINCT p.id_pedido)` es indispensable: como cada pedido tiene varias filas en `detalle_pedido`, `COUNT(*)` contaría productos, no pedidos. Verificar qué representa cada fila después de un JOIN es una de las buenas prácticas más importantes.

3.6 GROUP BY y HAVING

HAVING filtra **grupos** después de agrupar, usando el resultado de una función de agregación. **WHERE**, en cambio, filtra **filas** antes de agrupar. Ambos pueden usarse en la misma consulta.

	WHERE	HAVING
Qué filtra	Filas individuales	Grupos ya formados
Cuándo actúa	Antes de GROUP BY	Después de GROUP BY
¿Admite COUNT, SUM, AVG...?	No	Sí
Ejemplo	WHERE estado = 'Entregado'	HAVING SUM(...) > 300

PREGUNTA DE NEGOCIO

¿Qué clientes superan S/ 300 en compras ya entregadas? (candidatos a un programa de fidelización)

CONSULTA SQL

```
SELECT c.nombre AS cliente,
       SUM(d.cantidad * d.precio_unitario) AS monto_total
FROM clientes AS c
INNER JOIN pedidos AS p ON c.id_cliente = p.id_cliente
INNER JOIN detalle_pedido AS d ON p.id_pedido = d.id_pedido
WHERE p.estado = 'Entregado'
GROUP BY c.nombre
HAVING SUM(d.cantidad * d.precio_unitario) > 300
ORDER BY monto_total DESC;
```

RESULTADO ESPERADO

cliente	monto_total
Ana Torres	475.00

INTERPRETACIÓN

Solo Ana Torres cumple la condición. En la consulta anterior, sin el filtro **WHERE**, tres clientes superaban S/ 300; pero los segundos pedidos de Jorge y de Luis (108 y 107) todavía están en camino. **WHERE** los excluye antes de sumar y **HAVING** conserva solo los grupos que superan el umbral.

Para la gerencia, la lección es que el criterio de negocio define el resultado: "clientes que más compraron" y "clientes con más compras entregadas" son preguntas distintas y producen listas distintas.

3.7 Subconsultas

Una **subconsulta** es una consulta escrita dentro de otra, entre paréntesis. Se utiliza cuando la condición depende de un valor que primero debe calcularse.

PREGUNTA DE NEGOCIO

¿Qué productos tienen un precio superior al promedio del catálogo?

CONSULTA SQL

```
SELECT nombre_producto, precio_unitario
FROM productos
WHERE precio_unitario > (SELECT AVG(precio_unitario)
                        FROM productos)
ORDER BY precio_unitario DESC;
```

RESULTADO ESPERADO

nombre_producto	precio_unitario
Teclado mecánico	210.00
Licuada 600 W	189.90
Audífonos inalámbricos	120.00

INTERPRETACIÓN

La subconsulta calcula el precio promedio del catálogo (S/ 107.49) y la consulta principal devuelve los productos que lo superan. El valor no se escribe a mano: si mañana cambian los precios, la consulta sigue siendo válida. Esta lista ayuda a identificar los productos de mayor valor, en los que un error de stock tiene más impacto.

PREGUNTA DE NEGOCIO

¿Qué productos del catálogo nunca se han vendido?

CONSULTA SQL

```
SELECT nombre_producto, precio_unitario
FROM productos
WHERE id_producto NOT IN (SELECT id_producto
                        FROM detalle_pedido);
```

RESULTADO ESPERADO

nombre_producto	precio_unitario
Libro Introducción a SQL	65.00

INTERPRETACIÓN

El libro "Introducción a SQL" no aparece en ningún pedido. Es el hallazgo que la tabla `ventas` no podía mostrar en la Semana 2. El área comercial puede revisar su precio, su visibilidad en la tienda o su permanencia en el catálogo. El mismo resultado se obtiene con un `LEFT JOIN` entre `productos` y `detalle_pedido` filtrando `IS NULL`.

3.8 Buenas prácticas de escritura SQL

Una consulta se escribe una vez, pero se lee y se corrige muchas veces. Compara la consulta de la sección 3.5 con esta versión en una sola línea: `select nombre,sum(cantidad*precio_unitario) from clientes c join pedidos p on ....` Ambas funcionan, pero solo la primera se revisa con facilidad. Todas las consultas de este material siguen estas prácticas:

- Palabras clave en MAYÚSCULAS, una cláusula por línea y comentarios con `--` para explicar la intención.
- Columnas nombradas en lugar de `SELECT *` en consultas definitivas; `AS` para dar nombres comprensibles a los cálculos.
- Alias cortos y columnas calificadas (`c.nombre`) cuando hay más de una tabla.
- Construcción por partes: primero el `JOIN`, luego filtros y agrupación, verificando el número de filas en cada paso; `LIMIT` al explorar.

3.9 De la necesidad de negocio a la consulta

Escribir SQL no empieza en el teclado, sino en la pregunta. Este procedimiento, aplicado a la consulta de fidelización de la sección 3.6, sirve para cualquier requerimiento:

Paso	Pregunta de control	Aplicación al caso de fidelización
1. Precisar la pregunta	¿Qué se quiere saber exactamente y para qué?	Clientes con más de S/ 300 en compras entregadas, para invitarlos a un programa
2. Identificar los datos	¿Qué columnas y tablas contienen la respuesta?	Nombre (clientes), estado (pedidos), cantidad y precio (detalle_pedido)
3. Seguir las relaciones	¿Qué claves conectan esas tablas?	<code>clientes.id_cliente = pedidos.id_cliente; pedidos.id_pedido = detalle_pedido.id_pedido</code>
4. Definir filtros y grupos	¿Qué filas filtro? ¿Cómo agrupo? ¿Qué grupos conservo?	<code>WHERE estado = 'Entregado'; GROUP BY cliente; HAVING monto > 300</code>
5. Validar e interpretar	¿El resultado es plausible? ¿Qué decisión permite?	Un cliente califica; se verifica a mano y se comunica con su criterio

Cierre de la Semana 3

Lo que debes dominar

- Explicar por qué una base de datos separa la información en varias tablas y qué problemas evita.
- Diferenciar clave primaria y clave foránea e identificarlas en un conjunto de tablas.
- Elegir entre `INNER JOIN` y `LEFT JOIN` según la pregunta, y encadenar `JOIN` de varias tablas.
- Usar `WHERE` para filtrar filas y `HAVING` para filtrar grupos, en la misma consulta si es necesario.
- Resolver preguntas con subconsultas y escribir SQL legible, partiendo siempre de una necesidad de negocio.

Comprueba tu aprendizaje

1. ¿Por qué `INNER JOIN` entre `clientes` y `pedidos` no permite encontrar a los clientes que nunca compraron?
2. Escribe una consulta que muestre cuántos pedidos tiene cada estado (Entregado, En camino).
3. Explica con tus palabras por qué la condición `SUM(total) > 300` no puede escribirse en el `WHERE`.
4. Después de un `JOIN` entre `pedidos` y `detalle_pedido`, un analista usa `COUNT(*)` para contar pedidos y obtiene 13 en lugar de 8. ¿Qué ocurrió?

Conexión con la Semana 4

Semana 3 → ya sabemos relacionar tablas y responder preguntas que combinan clientes, pedidos y productos.

Semana 4 → ahora nos preguntaremos *por qué* esas tablas tienen esa forma: cómo se identifican las entidades, cómo se definen sus relaciones y qué reglas (normalización) llevan de la tabla `ventas` a las cinco tablas de RutaMarket.

Semana 4 · Modelado Relacional

Pregunta guía: ¿cómo se diseñan las tablas y sus relaciones para que los datos sean coherentes y fáciles de consultar?

En la Semana 3 usamos un conjunto de tablas ya diseñado. En la práctica, alguien tuvo que decidir qué tablas crear, qué columnas poner en cada una y cómo conectarlas. Esa tarea es el **modelado**. Así como una casa se construye a partir de un plano, una base de datos se construye a partir de un modelo: corregir el plano es sencillo; corregir la casa construida, no.

4.1 El modelo relacional

Concepto. El **modelo relacional**, propuesto por Edgar F. Codd (1970), organiza los datos en **relaciones** (tablas). Cada fila se identifica por una clave y las tablas se vinculan mediante los valores de esas claves, no mediante punteros o posiciones físicas.

Aplicación. Entender el modelo permite saber qué tablas unir, por qué columnas y qué representa cada fila; es decir, escribir consultas correctas como las de la Semana 3 y detectar cuándo un resultado no tiene sentido.

4.2 Entidades y atributos

Concepto. Una **entidad** es un objeto o concepto del negocio sobre el que se necesita guardar datos; se convierte en una tabla. Un **atributo** es una característica de la entidad; se convierte en una columna.

Ejemplo. Una técnica sencilla consiste en analizar la descripción del negocio: los **sustantivos** sugieren entidades o atributos y los **verbos**, relaciones. *"Los clientes realizan pedidos. Cada pedido incluye uno o varios productos. Cada producto pertenece a una categoría."*

Entidad	Qué representa	Atributos principales
Cliente	Persona que compra en la tienda	id_cliente, nombre, ciudad, email, fecha_registro
Pedido	Una compra realizada en una fecha	id_pedido, fecha_pedido, estado
Producto	Artículo del catálogo	id_producto, nombre_producto, precio_unitario
Categoría	Agrupación de productos	id_categoria, nombre_categoria

Interpretación. Un atributo debe describir **solo** a su entidad. La ciudad describe al cliente, no al producto; el precio de catálogo describe al producto, no al cliente. Cuando un atributo parece pertenecer a dos entidades a la vez (como la cantidad comprada, que depende del pedido y del producto), suele indicar que falta una entidad intermedia.

4.3 Claves primarias y foráneas en el diseño

En la Semana 3 usamos las claves para unir tablas; en el diseño hay que **elegirlas**. Una buena clave primaria es **única**, **nunca está vacía** y es **estable** (no cambia con el tiempo). Para **clientes**, el nombre no sirve (dos personas pueden llamarse igual) y el correo es único pero puede cambiar; por eso se usa **id_cliente**, un código generado por el sistema.

Cuando una sola columna no basta para identificar una fila, se usa una **clave primaria compuesta**. En **detalle_pedido**, la pareja (**id_pedido**, **id_producto**) identifica cada fila: un mismo producto no se registra dos veces en el mismo pedido. Cada una de esas columnas es, a la vez, clave foránea hacia su tabla de origen.

4.4 Relaciones y cardinalidad

Concepto. La **cardinalidad** indica cuántas filas de una entidad pueden relacionarse con cuántas filas de otra. Determina dónde se coloca la clave foránea o si se necesita una tabla adicional.

	1:1 (uno a uno)	1:N (uno a muchos)	N:M (muchos a muchos)
Significado	Cada fila de A se relaciona con una sola fila de B, y viceversa	Una fila de A se relaciona con muchas de B; cada fila de B, con una de A	Muchas filas de A se relacionan con muchas de B
Ejemplo en RutaMarket	Cliente ↔ cuenta de acceso (usuario y contraseña)	Cliente → pedidos; categoría → productos	Pedidos ↔ productos
Cómo se implementa	FK con valores únicos en una de las tablas (o ambas en una sola tabla)	FK en la tabla del lado "muchos"	Tabla intermedia con dos FK

Tablas intermedias. Una relación N:M no puede representarse con una sola FK: un pedido tendría que guardar varios productos en una celda. Se crea entonces una **tabla intermedia**, `detalle_pedido`, con una FK hacia cada tabla. La tabla intermedia también guarda los atributos de la relación: la **cantidad** y el **precio unitario al momento de la venta**. Este último no es una repetición del precio de catálogo: si el precio cambia en abril, los pedidos de marzo deben conservar el precio que realmente se cobró.

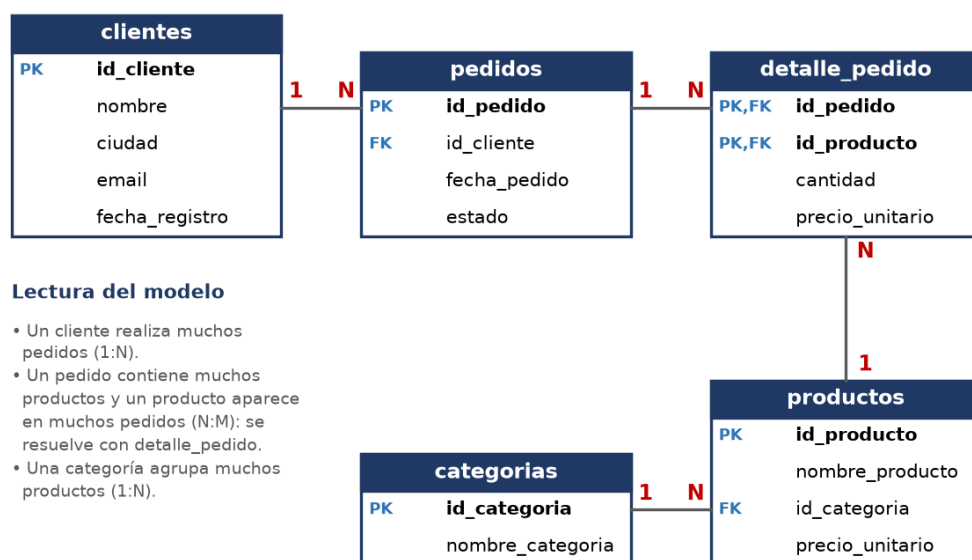


Figura 3. Modelo relacional de RutaMarket (PK: clave primaria; FK: clave foránea).

Interpretación. El diagrama se lee siguiendo las líneas: el "1" y la "N" indican la cardinalidad. Por ejemplo, entre `clientes` y `pedidos`, un cliente puede tener muchos pedidos, y cada pedido pertenece a un solo cliente. Los caminos del diagrama son los mismos que seguimos al escribir los JOIN de la Semana 3.

4.5 Normalización: 1FN, 2FN y 3FN

Concepto. La **normalización** es un proceso de revisión del diseño que busca que cada dato se guarde en un solo lugar. Se realiza por etapas llamadas **formas normales**; cada una elimina un tipo de redundancia (Codd, 1970; Elmasri y Navathe, 2016).

Contexto. Una tabla mal diseñada produce **anomalías**:

Anomalía	Qué ocurre	Ejemplo con la tabla ventas
De actualización	Un cambio debe hacerse en muchas filas; si se olvida una, hay contradicción	Ana se muda y debe corregirse su ciudad en cuatro filas
De inserción	No se puede registrar un dato sin otro que aún no existe	No se puede registrar un producto nuevo hasta que se venda
De eliminación	Al borrar un dato se pierde otro que se quería conservar	Si se anula la única venta de Rosa Flores, se pierde su registro como cliente

Punto de partida: la planilla de pedidos

Supongamos que RutaMarket empezó registrando sus pedidos en una hoja de cálculo como esta:

id_pedido	fecha	cliente	ciudad	productos
101	2026-03-02	Ana Torres	Lima	Audífonos inalámbricos (1), Mouse inalámbrico (2)
102	2026-03-05	Luis Quispe	Arequipa	Licudadora 600 W (1)

Primera Forma Normal (1FN): valores atómicos

Regla: cada celda contiene un único valor y no hay grupos repetidos (ni listas en una celda ni columnas como producto1, producto2...). La columna `productos` viola esta regla: no es posible filtrar, contar ni sumar por producto. Al separar cada producto en su propia fila obtenemos una tabla con clave compuesta (`id_pedido`, `id_producto`), con columnas como fecha, cliente, ciudad, producto, categoría, precio y cantidad. **Esa tabla es, en esencia, la tabla `ventas` de la Semana 2:** está en 1FN, pero aún repite datos.

Segunda Forma Normal (2FN): sin dependencias parciales

Regla: estar en 1FN y que cada atributo que no forma parte de la clave dependa de la **clave completa**, no de una parte de ella. En la tabla anterior, la fecha y el cliente dependen solo de `id_pedido`; el nombre, la categoría y el precio de catálogo dependen solo de `id_producto`; únicamente la cantidad (y el precio cobrado) dependen de ambos. Se separa en tres tablas: `pedidos` (`id_pedido`, fecha, cliente, ciudad), `productos` (`id_producto`, nombre, categoría, precio) y `detalle_pedido` (`id_pedido`, `id_producto`, cantidad, precio_unitario).

Tercera Forma Normal (3FN): sin dependencias transitivas

Regla: estar en 2FN y que ningún atributo que no es clave dependa de otro atributo que tampoco es clave. En `pedidos`, la ciudad depende del cliente, no del pedido (`id_pedido` → cliente → ciudad): se crea la tabla `clientes` y en `pedidos` queda solo `id_cliente` como FK. Del mismo modo, la categoría se describe en su propia tabla `categorias` y en `productos` queda `id_categoria`. El resultado son las **cinco tablas** de la Figura 3.

	1FN	2FN	3FN
Pregunta de verificación	¿Cada celda tiene un solo valor?	¿Cada atributo depende de toda la clave?	¿Algún atributo depende de otro que no es clave?
Problema que elimina	Listas y grupos repetidos	Datos que dependen de parte de una clave compuesta	Datos que dependen de otros datos no clave
Ejemplo en RutaMarket	"Audífonos (1), Mouse (2)" en una celda	Nombre de producto repetido en cada venta	Ciudad del cliente repetida en cada pedido
Solución	Una fila por producto del pedido	Separar pedidos, productos y detalle	Separar clientes y categorías

Interpretación. Normalizar no significa "crear muchas tablas", sino que **cada hecho se registre una sola vez**. Así, la ciudad de Ana se corrige en una sola fila, un producto puede existir antes de venderse y un cliente se conserva aunque se anule su pedido. Las tablas cómodas para consultar, como `ventas`, se obtienen cuando se necesitan mediante JOIN.

4.6 Del modelo a las tablas en SQLite

El modelo se implementa con la instrucción `CREATE TABLE`. Las **restricciones** convierten las reglas del diseño en controles que la base de datos aplica automáticamente. Este fragmento define dos tablas del modelo de RutaMarket (las demás se crean de la misma forma):

```
PRAGMA foreign_keys = ON;  -- en SQLite, activa el control de claves foráneas

CREATE TABLE pedidos (
    id_pedido    INTEGER PRIMARY KEY,
    id_cliente   INTEGER NOT NULL REFERENCES clientes(id_cliente),
    fecha_pedido TEXT    NOT NULL,
    estado       TEXT     NOT NULL CHECK (estado IN ('Entregado', 'En camino', 'Anulado'))
);

CREATE TABLE detalle_pedido (
    id_pedido    INTEGER NOT NULL REFERENCES pedidos(id_pedido),
    id_producto  INTEGER NOT NULL REFERENCES productos(id_producto),
    cantidad     INTEGER NOT NULL CHECK (cantidad > 0),
    precio_unitario REAL  NOT NULL CHECK (precio_unitario >= 0),
    PRIMARY KEY (id_pedido, id_producto)
);
```

Interpretación. Con estas definiciones, la base de datos rechaza un pedido de un cliente inexistente (`REFERENCES`), una fila de detalle duplicada (`PRIMARY KEY`), un pedido sin fecha (`NOT NULL`), una cantidad

negativa o un estado no previsto (**CHECK**). En SQLite, el control de claves foráneas debe activarse con `PRAGMA foreign_keys = ON`.

4.7 Errores frecuentes de diseño

Error	Consecuencia	Corrección
Varios valores en una celda ("Audífonos, Mouse")	No se puede filtrar, contar ni sumar por valor	Aplicar 1FN: una fila por valor
Columnas repetidas (producto1, producto2, producto3)	Límite arbitrario y consultas complicadas	Tabla intermedia (relación N:M)
Tabla sin clave primaria	Filas duplicadas imposibles de distinguir	Definir una PK única y estable
Usar el nombre como clave	Homónimos y cambios de nombre rompen las relaciones	Usar un identificador generado
Repetir datos descriptivos (nombre y ciudad del cliente en cada pedido)	Anomalías de actualización e inconsistencias	Aplicar 2FN y 3FN
Fechas como texto en formatos mezclados	Filtros y ordenamientos incorrectos	Formato único (AAAA-MM-DD) y validación

4.8 Buen modelo, mejores datos

En la Semana 1 estudiamos las dimensiones de calidad. Un buen modelo relacional protege varias de ellas **desde el origen**, en lugar de corregirlas después:

Dimensión de calidad	Mecanismo del modelo que la protege
Unicidad	Clave primaria y restricción UNIQUE (por ejemplo, en el correo)
Compleitud	NOT NULL en los atributos obligatorios
Consistencia	Normalización: cada dato se guarda en un solo lugar; claves foráneas que impiden referencias huérfanas
Validez	Tipos de datos adecuados y reglas CHECK (cantidad > 0, estados permitidos)

Interpretación. El modelo no lo resuelve todo: nada impide registrar "Arequpa" si la columna acepta cualquier texto. La **exactitud** y la **oportunidad** dependen también de los procesos de captura y de las personas. Por eso el diseño de datos y la gestión de datos se complementan: el modelo fija las reglas y la gestión vela por que se cumplan.

Cierre de la Semana 4

Lo que debes dominar

- Identificar entidades, atributos y relaciones a partir de la descripción de un negocio.
- Elegir claves primarias adecuadas (únicas, obligatorias, estables) y ubicar correctamente las claves foráneas.
- Reconocer cardinalidades 1:1, 1:N y N:M, y resolver las N:M con una tabla intermedia.
- Aplicar 1FN, 2FN y 3FN para eliminar redundancias y anomalías.
- Explicar cómo las restricciones del modelo (PK, FK, NOT NULL, UNIQUE, CHECK) protegen la calidad de los datos.

Comprueba tu aprendizaje

1. ¿Por qué el precio unitario se guarda en `detalle_pedido` si ya existe en `productos`? ¿Es una redundancia?
2. Una universidad registra estudiantes y cursos; un estudiante lleva varios cursos y un curso tiene varios estudiantes. Identifica la cardinalidad y propone las tablas necesarias con sus PK y FK.
3. La tabla (`id_pedido`, `id_producto`, `cantidad`, `nombre_producto`) tiene clave compuesta (`id_pedido`, `id_producto`). ¿Qué forma normal incumple y por qué?
4. Menciona dos errores de calidad que el modelo puede impedir y uno que no puede impedir.

Integración de la Unidad 1

Del registro de un hecho a una estructura relacional capaz de responder preguntas de negocio.

Las cuatro semanas forman un solo recorrido. Cada etapa depende de la anterior: no se puede consultar lo que no se ha organizado, ni relacionar tablas que no se diseñaron con claves, ni confiar en un resultado cuyos datos de origen tienen mala calidad.

Mapa conceptual del recorrido

Etapas	Idea clave	Semana	Pregunta que responde
DATO	Hecho registrado; necesita contexto y calidad	1	¿Qué se registró?
→ INFORMACIÓN	Datos organizados y resumidos que responden preguntas	1–2	¿Qué ocurrió? ¿Cuánto? ¿Dónde?
→ BASE DE DATOS	Tablas con filas, columnas y claves, administradas por un SGBD	2	¿Dónde y cómo se guardan los datos?
→ SQL	Lenguaje para filtrar, ordenar, agregar y agrupar	2	¿Cómo obtengo una respuesta precisa?
→ RELACIONES	PK, FK y JOIN para combinar tablas	3	¿Cómo combino datos de distintas tablas?
→ MODELO RELACIONAL	Entidades, cardinalidad y normalización	4	¿Cómo diseño tablas coherentes y sin redundancia?

Caso integrador

La gerencia de RutaMarket evalúa abrir un **punto de recojo** en la ciudad que concentre más pedidos e ingresos. Resolver este requerimiento moviliza todo lo aprendido en la unidad:

1. ¿Qué datos necesita la organización? La ciudad de cada cliente, sus pedidos y el detalle de cada pedido (cantidad y precio). Son datos **estructurados**. Antes de calcular, hay que verificar su calidad: si una ciudad aparece escrita de dos formas ("Lima" y "lima"), los resultados se dividirán en grupos distintos (Semana 1).

2. ¿Cómo deberían organizarse? La ciudad es un atributo del cliente y se guarda una sola vez en `clientes`; los pedidos, en `pedidos`; las cantidades y precios, en `detalle_pedido` (Semanas 2 y 4).

3. y 4. ¿Cómo consultarlos y relacionarlos? Se sigue el camino de claves `clientes` → `pedidos` → `detalle_pedido` y se agrupa por ciudad (Semanas 2 y 3).

PREGUNTA DE NEGOCIO

¿Qué ciudades concentran más pedidos e ingresos?

CONSULTA SQL

```
SELECT c.ciudad,
       COUNT(DISTINCT p.id_pedido) AS pedidos,
       SUM(d.cantidad * d.precio_unitario) AS ingresos
FROM clientes AS c
INNER JOIN pedidos AS p ON c.id_cliente = p.id_cliente
INNER JOIN detalle_pedido AS d ON p.id_pedido = d.id_pedido
GROUP BY c.ciudad
ORDER BY ingresos DESC;
```

RESULTADO ESPERADO

ciudad	pedidos	ingresos
Lima	4	984.90
Arequipa	2	444.90
Cusco	1	215.00
Trujillo	1	190.00

INTERPRETACIÓN

Lima concentra 4 de los 8 pedidos y S/ 984.90 de S/ 1,834.80 (alrededor del 54 % de los ingresos de marzo). Es la candidata natural para el punto de recojo. Sin embargo, la cifra proviene de un solo mes y de solo dos clientes limeños: antes de invertir, la gerencia debería confirmar el patrón con más periodos.

5. ¿El diseño relacional es adecuado para esta decisión? Aquí aparece una reflexión de diseño: el modelo guarda la ciudad de **residencia** del cliente, pero un punto de recojo depende de la ciudad de **entrega** de cada pedido, que podría ser distinta. Si ese dato es importante para el negocio, el modelo debería incorporar una dirección o ciudad de entrega en **pedidos**. Un buen modelo no solo está normalizado: también contiene los datos que las preguntas del negocio requieren (Semana 4).

Checklist de dominio de la Unidad 1

Marca cada afirmación solo si puedes demostrarla con un ejemplo propio:

- ☐ Explico la diferencia entre dato, información y conocimiento con un ejemplo de una organización.
- ☐ Clasifico datos en estructurados, semiestructurados y no estructurados y sé cuáles se consultan directamente con SQL.
- ☐ Describo el ciclo de vida del dato y reconozco en qué etapas se originan los problemas de calidad.
- ☐ Identifico problemas de calidad en una tabla y los asocio a la dimensión correspondiente.
- ☐ Escribo consultas con SELECT, WHERE, ORDER BY, LIMIT y DISTINCT, y explico qué representa cada fila del resultado.
- ☐ Resumo datos con COUNT, SUM, AVG, MAX, MIN y GROUP BY, e interpreto los resultados para una persona que toma decisiones.
- ☐ Distingo clave primaria de clave foránea y elijo entre INNER JOIN y LEFT JOIN según la pregunta.
- ☐ Uso WHERE y HAVING en la misma consulta y resuelvo preguntas con subconsultas.
- ☐ Identifico entidades, atributos y cardinalidades (1:1, 1:N, N:M) y resuelvo una relación N:M con una tabla intermedia.
- ☐ Normalizo una tabla hasta 3FN y explico cómo el modelo protege la calidad de los datos.

Conclusiones

1. Un dato solo adquiere valor cuando tiene contexto, calidad y un propósito. La jerarquía DIKW recuerda que el objetivo de la gestión de datos no es acumular registros, sino convertirlos en información y conocimiento que orienten decisiones.
2. La calidad del dato se decide en todo el ciclo de vida, especialmente en la captura y el almacenamiento. Ninguna consulta ni herramienta posterior puede corregir por completo datos mal registrados.
3. SQL permite pasar de los datos a la información de forma precisa y reproducible. Su dominio implica tanto escribir consultas correctas como interpretar sus resultados: qué representa cada fila, qué filtros se aplicaron y qué definición de cada métrica se utilizó.
4. Las bases de datos relacionales distribuyen la información en tablas conectadas por claves. Los JOIN permiten volver a reunirla según la pregunta, y la elección entre INNER y LEFT JOIN, o entre WHERE y HAVING, cambia la respuesta.
5. El modelado relacional y la normalización explican por qué las tablas tienen la forma que tienen: cada hecho se registra una sola vez, lo que evita anomalías y protege la calidad de los datos desde el diseño.
6. Estos fundamentos son la base para los temas siguientes del curso y para el trabajo en Big Data y Ciencia de Datos: antes de preparar o analizar datos es necesario saber dónde están, cómo están organizados y cómo obtenerlos correctamente.

Referencias bibliográficas

- Ackoff, R. L. (1989). From data to wisdom. *Journal of Applied Systems Analysis*, 16, 3–9.
- Beaulieu, A. (2020). *Learning SQL: Generate, manipulate, and retrieve data* (3.^a ed.). O'Reilly Media.
- Chamberlin, D. D., y Boyce, R. F. (1974). SEQUEL: A structured English query language. En *Proceedings of the 1974 ACM SIGFIDET Workshop on Data Description, Access and Control* (pp. 249–264). ACM.
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. <https://doi.org/10.1145/362384.362685>

- Congreso de la República del Perú. (2011). *Ley N.º 29733, Ley de Protección de Datos Personales*. Diario Oficial El Peruano.
- Coronel, C., y Morris, S. (2019). *Database systems: Design, implementation, and management* (13.ª ed.). Cengage Learning.
- DAMA International. (2017). *DAMA-DMBOK: Data management body of knowledge* (2.ª ed.). Technics Publications.
- Date, C. J. (2019). *Database design and relational theory: Normal forms and all that jazz* (2.ª ed.). Apress.
- Elmasri, R., y Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.
- Laney, D. (2001). *3D data management: Controlling data volume, velocity and variety*. META Group.
- Provost, F., y Fawcett, T. (2013). *Data science for business*. O'Reilly Media.
- Rowley, J. (2007). The wisdom hierarchy: Representations of the DIKW hierarchy. *Journal of Information Science*, 33(2), 163–180. <https://doi.org/10.1177/0165551506070706>
- SQLite. (s. f.). *SQLite documentation*. <https://www.sqlite.org/docs.html>
- Wang, R. Y., y Strong, D. M. (1996). Beyond accuracy: What data quality means to data consumers. *Journal of Management Information Systems*, 12(4), 5–33.